Laboratoire 3 - Affichage de l'heure sur LCD avec TIM2

Matisse Rhéaume-Viale

6 octobre 2026



Techniques de génie électronique programmable

Cours : 243-356-LI - Programmation 3 - Microcontrôleurs et périphériques
Enseignant : Gabriel Fortin-Bélanger
Groupe : 0001

0. Introduction

Dans ce laboratoire, j'utilise un STM32F103 pour afficher l'heure sur un écran LCD 20 × 4 au format `HH:MM:SS`. Le programme utilise TIM2 pour générer une interruption chaque seconde et faire avancer l'heure automatiquement.

L'écran LCD communique avec le STM32 en mode 4 bits à l'aide des broches PB9 à PB15. Le programme utilise directement les registres du microcontrôleur, sans bibliothèque HAL. SysTick est aussi utilisé pour gérer les délais nécessaires aux commandes du LCD.

Le fonctionnement est assez simple : TIM2 met à jour l'heure chaque seconde, puis la boucle principale détecte le changement et actualise l'affichage.

0.1 Schéma fonctionnel

Le schéma suivant représente le fonctionnement général du programme et les échanges entre les différentes parties.



Figure 1 - Schéma fonctionnel du laboratoire 3.

1. Objectif

Le but est d'afficher l'heure au format `HH:MM:SS` sur l'écran LCD. Le Timer 2 déclenche une interruption chaque seconde. L'interruption avance l'heure et la boucle principale met l'écran à jour.

Le programme utilise les registres du STM32F103 directement. Il n'utilise pas HAL. L'écran est piloté en mode 4 bits avec les broches `DB4` à `DB7`.

2. Branchement de l'écran

Signal LCD	Broche STM32F103	Rôle
RS	PB9	Sélection commande ou donnée
R/W	PB10	Sélection lecture ou écriture
E	PB11	Validation du transfert
DB4	PB12	Donnée, bit 4
DB5	PB13	Donnée, bit 5
DB6	PB14	Donnée, bit 6
DB7	PB15	Donnée, bit 7



INTERFACE PIN FUNCTIONS

Pin No.	Symbol	Level	Description
1	VSS	0V	Ground.
2	VDD	+5.0V	Power supply for logic operating.
3	V0	--	Adjusting supply voltage for LCD driving.
4	RS	H/L	A signal for selecting registers: 1: Data Register (for read and write) 0: Instruction Register (for write), Busy flag-Address Counter (for read).
5	R/W	H/L	R/W = "H": Read mode. R/W = "L": Write mode.
6	E	H/L	An enable signal for writing or reading data.
7	DB0	H/L	This is an 8-bit bi-directional data bus.
8	DB1	H/L	
9	DB2	H/L	
10	DB3	H/L	
11	DB4	H/L	
12	DB5	H/L	
13	DB6	H/L	
14	DB7	H/L	
15	LED+	+5.0V	Power supply for backlight.
16	LED-	0V	The backlight ground.

Figure 2 - Brochage du module TC2004A. Tinsharp, p. 5.

Le code écrit seulement dans l'écran. Il garde donc `R/W` à 0. La fonction `LCD_Commande()` ne fait rien si `RW` vaut `true`, car son prototype ne permet pas de retourner la valeur lue et les lignes de données sont configurées en sorties.

3. Fonctionnement du programme

Au démarrage, `SystemInit()` sélectionne le HSI à 8 MHz et laisse les préscalers AHB et APB à 1. `LCD_Init()` configure les broches et démarre l'écran en mode 4 bits. Dans `main()`, j'initialise ensuite l'heure à `23:59:55` avec `Timer2_SetTime()`, je l'affiche, puis je démarre TIM2. Cette valeur sert à préparer le passage à minuit.

TIM2 compte une seconde par période. Son interruption met à jour le nombre de secondes depuis minuit, augmente le compteur de mises à jour et envoie l'événement `SEV`. Cet événement ne sert pas à réveiller `main()`, car cette fonction ne contient pas de `WFE`. Elle ne contient pas non plus de boucle d'attente en C. Elle relit le compteur de mises à jour en continu et rafraîchit l'écran quand celui-ci change. Les écritures LCD restent dans le programme principal et ne sont pas faites dans l'interruption.

Le temps est gardé sous forme d'un nombre de secondes entre 0 et 86 399. Après `23:59:59`, il revient à zéro. Cette représentation rend le passage à la minute et à minuit simple à gérer.

Le fichier `gpio.c` contient aussi les fonctions pour la LED et le bouton, mais `main()` ne les appelle pas. `LED_Init()` configure PB6 en sortie push-pull à 2 MHz. `LED_Set()` commande la LED en actif bas. `Button_Init()` configure PB5 en entrée flottante et `Button_Read()` retourne 1 lorsque PB5 est bas. Les options de polarité et de pull interne qui étaient dans `gpio.h` ont été retirées ; ces réglages sont maintenant écrits directement dans le code.

Le fichier `SysTick.c` contient une base de temps de 1 ms, mais `main()` n'appelle pas `SysTick_Init()`. `LCD_Init()` configure plutôt SysTick comme compteur libre pour les délais du LCD.

4. Plan de test

J'ai vérifié les trois cas avec l'analyseur logique Saleae Logic 2 et le décodeur HD44780. Les captures montrent les signaux envoyés sur le bus LCD. Elles ne confirment pas à elles seules que l'écran affiche visuellement les caractères.

Cas	Vérification	Résultat attendu	Résultat obtenu
1	Capturer une commande LCD avec l'analyseur logique	Pour `0x28`, `RS=0`, `R/W=0`, puis les nibbles `0010` et `1000`, chacun validé par une impulsion sur `E`.	Réussi : `0x28` reconnu par le décodeur HD44780.
2	Mesurer la période entre les rafraîchissements	Les commandes de début de rafraîchissement sont espacées d'environ 1 s.	11 intervalles : minimum de 998,538 ms, moyenne de 998,802 ms et maximum de 1 001,097 ms.
3	Vérifier le retour à zéro	En partant de `23:59:55`, l'heure revient à `00:00:00` après cinq interruptions d'une seconde.	Passage de `23:59:59` à `00:00:00` observé ; intervalle mesuré de 998,538 ms.

4.1 Résultats détaillés des cas de test

Cas 1 - Transmission d'une commande. J'ai utilisé Saleae Logic 2 avec le décodeur HD44780 pour vérifier les commandes envoyées au LCD.

La capture montre la commande `0x28`, qui configure l'écran en mode 4 bits, avec deux lignes logiques et des caractères de 5×8 points. Les signaux `RS` et `R/W` sont à 0, ce qui indique une commande envoyée en écriture. Deux impulsions sur `E` transmettent les deux nibbles. Le décodeur reconnaît `0x28` ; le test est réussi.

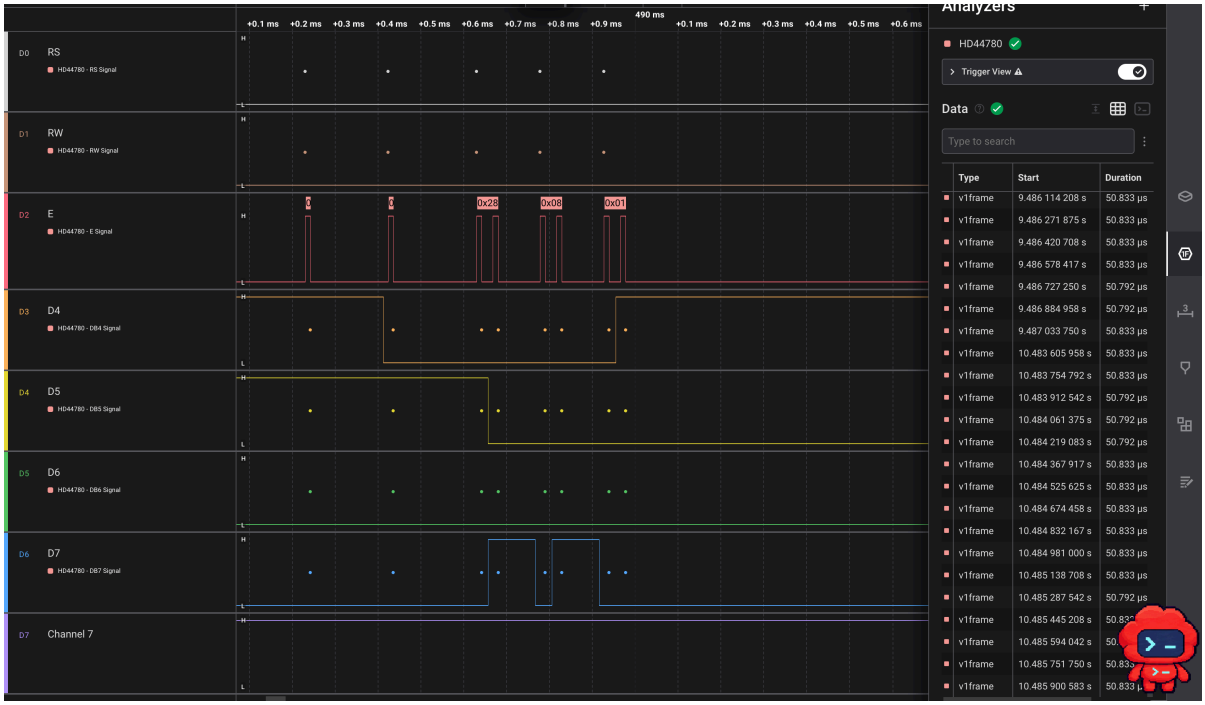


Figure 3 - Capture de Saleae Logic 2 montrant la commande 0x28 décodée par le décodeur HD44780.

Cas 2 - Intervalle entre les rafraîchissements. J'ai comparé les horodatages des commandes `0x80`, qui placent le curseur au début de la première ligne avant l'écriture de l'heure.

Sur les 11 intervalles mesurés, la moyenne est de 998,802 ms. Le minimum est de 998,538 ms et le maximum de 1 001,097 ms. La période théorique est de 1 000 ms. L'écart moyen est donc de -1,198 ms, soit environ -0,120 %. La capture montre des mises à jour environ chaque seconde, mais elle ne permet pas de confirmer la cause exacte de l'écart.

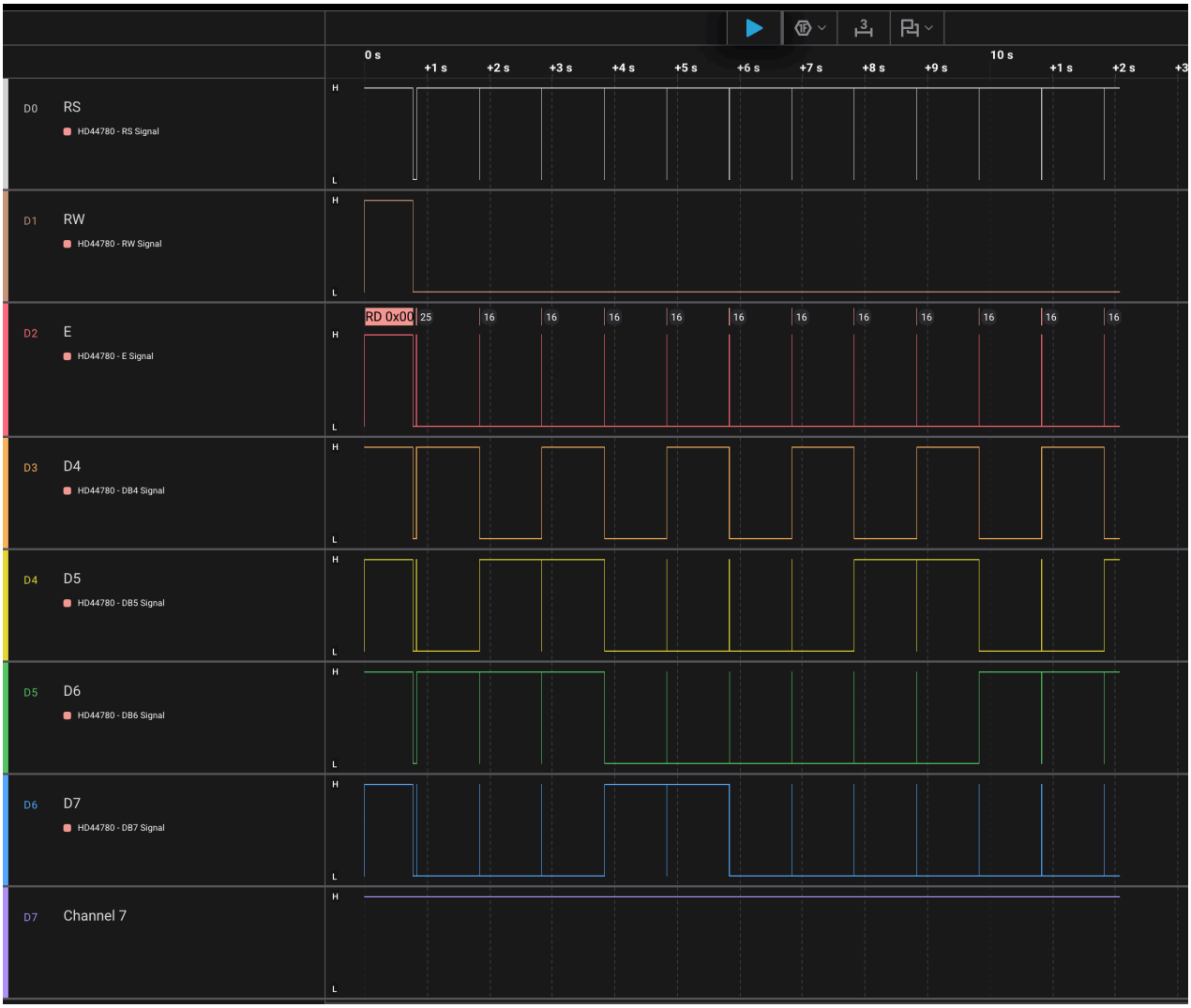


Figure 4 - Capture de Saleae Logic 2 utilisée pour mesurer les intervalles entre les rafraîchissements.

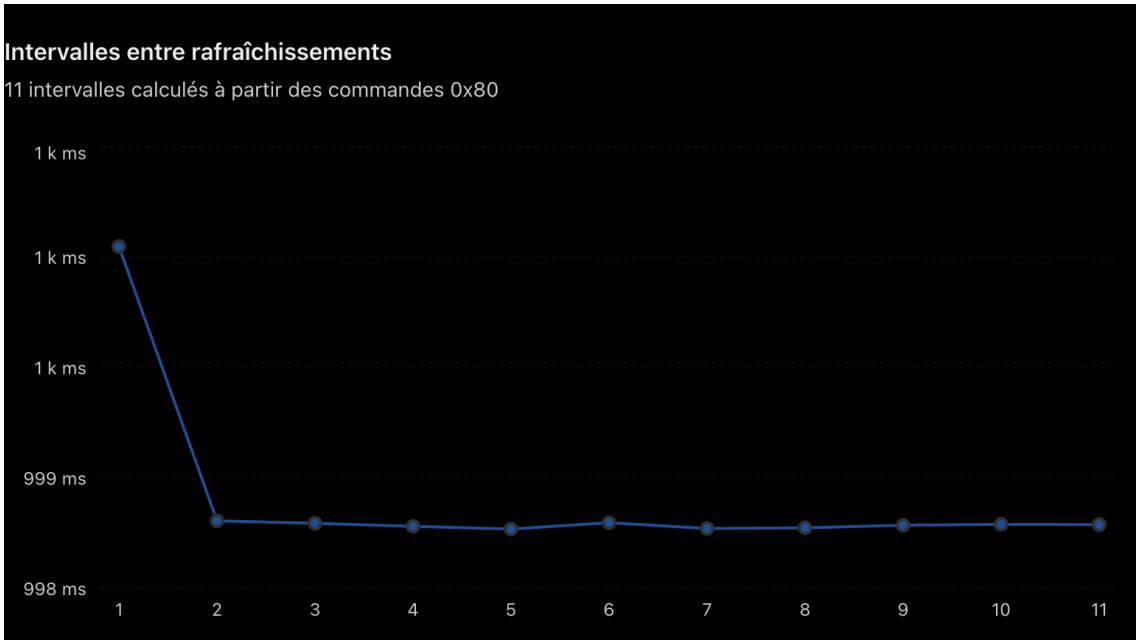


Figure 5 - Graphique des 11 intervalles mesurés entre les commandes de rafraîchissement.

Cas 3 - Passage de minuit. Pour ce test, le programme part de `23:59:55`. Dans l'enregistrement Saleae, j'ai observé les commandes correspondant à `23:59:59`, puis à `00:00:00`. Le dernier rafraîchissement de `23:59:59` commence à 4,831322 s et celui de `00:00:00` à 5,829860 s. L'intervalle mesuré est de 998,538 ms. Le passage à minuit est confirmé sur les signaux capturés.



Figure 6 - Résumé du passage de `23:59:59` à `00:00:00` observé au cas 3.

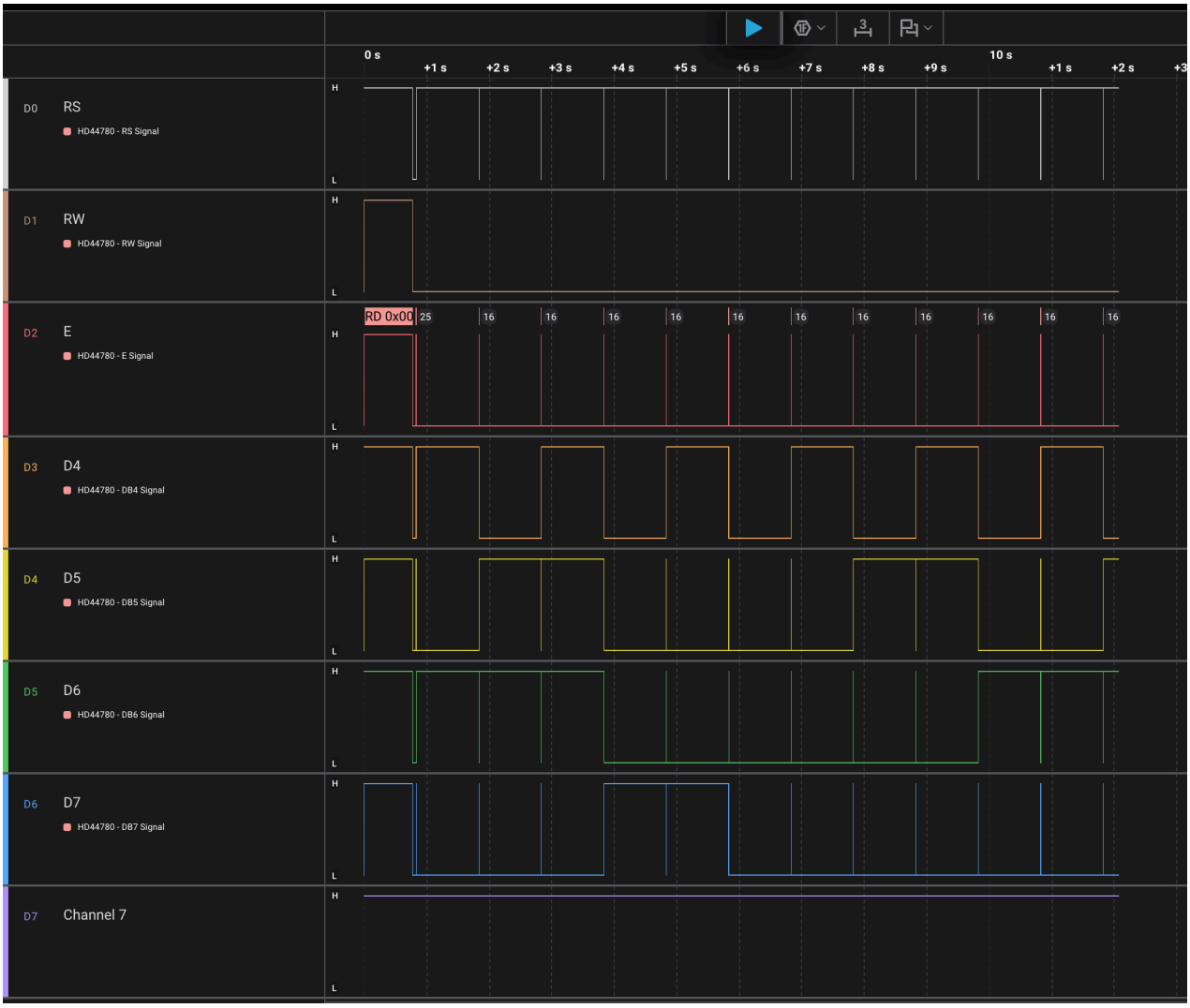


Figure 7 - Capture de Saleae Logic 2 utilisée pour mesurer les intervalles entre les rafraîchissements.

5. Analyse des fonctions demandées

5.1 LCD_Init()

Extrait du code source

```
void LCD_Init(void)
{
    LCD_GPIO_Init();

    /* SysTick sert ici de compteur libre pour les délais du LCD. */
    SYST_CSR = 0UL;
    SYST_RVR = SYST_RELOAD;
    SYST_CVR = 0UL;
    SYST_CSR = SYST_CLKSOURCE | SYST_ENABLE;

    LCD_Delai_Ms(LCD_STARTUP_DELAY_MS); // laisse le temps au LCD de démarrer

    /* Séquence spéciale: au démarrage, le contrôleur attend encore 8 bits. */
    LCD_Envoyer_Nibble(LCD_INIT_8BIT);
    LCD_Delai_Us(LCD_INIT_DELAY1_US);

    LCD_Envoyer_Nibble(LCD_INIT_8BIT);
    LCD_Delai_Ms(LCD_INIT_DELAY_MS);

    LCD_Envoyer_Nibble(LCD_INIT_8BIT);
    LCD_Delai_Us(LCD_INIT_DELAY2_US);

    LCD_Envoyer_Nibble(LCD_INIT_4BIT); // passe en mode 4 bits
    LCD_Delai_Us(LCD_INIT_DELAY2_US);

    LCD_Commande(false, false, LCD_CMD_FUNCTION_SET); // 4 bits, 2 lignes, 5x8
    LCD_Commande(false, false, LCD_CMD_DISPLAY_OFF);
    LCD_Commande(false, false, LCD_CMD_CLEAR);
    LCD_Commande(false, false, LCD_CMD_ENTRY_MODE);
    LCD_Commande(false, false, LCD_CMD_DISPLAY_ON);
}
```

Explication

`LCD_GPIO_Init()` active l'horloge de GPIOB, configure PB9 à PB15 comme sorties push-pull et met les sorties à zéro. SysTick est démarré sans interruption ; il sert de compteur libre pour mesurer les délais. À 8 MHz, chaque microseconde correspond à 8 cycles.

Après 50 ms, le code envoie les nibbles `0011`, `0011`, `0011`, puis `0010`. Cette séquence place le contrôleur LCD en mode 4 bits. Les commandes suivantes configurent l'écran, effacent son contenu et activent l'affichage. `0x28` choisit le transfert 4 bits et le format de caractère 5×8 points.

Registre	Adresse	Utilisation dans le pilote
`RCC_APB2ENR`	`0x40021018`	Bit 3 à 1 pour activer GPIOB.
`GPIOB_CRH`	`0x40010C04`	Configure PB9 à PB15 en sorties push-pull à 2 MHz (`MODE=10`, `CNF=00`).
`GPIOB_BSRR`	`0x40010C10`	Met les broches de commande et de données à 0 sans opération de lecture-modification-écriture.
`SysTick_CSR`	`0xE000E010`	Sélectionne l'horloge processeur et démarre le compteur ; `TICKINT` reste à 0.
`SysTick_RVR`	`0xE000E014`	Fixe le rechargement à la valeur maximale de 24 bits.
`SysTick_CVR`	`0xE000E018`	Fournit la valeur descendante utilisée pour calculer les délais.

Les extraits du manuel STM32 montrent le bit d'activation de GPIOB, la configuration de `GPIOx_CRH` et le registre de sortie `GPIOx_BSRR`.

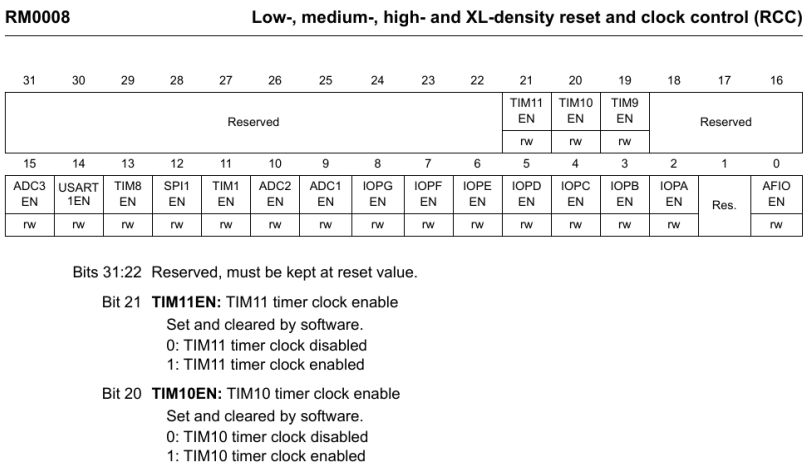


Figure 8 - Registre de configuration et activation de GPIOB. ST RM0008, p. 113.

9.2.2 Port configuration register high (GPIOx_CRH) (x=A..G)

Address offset: 0x04

Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30, 27:26, 23:22, 19:18, 15:14, 11:10, 7:6, 3:2 **CNFy[1:0]:** Port x configuration bits (y= 8 .. 15)
These bits are written by software to configure the corresponding I/O port.
Refer to *Table 20: Port bit configuration table*.
In input mode (MODE[1:0]=00):
00: Analog mode
01: Floating input (reset state)
10: Input with pull-up / pull-down
11: Reserved
In output mode (MODE[1:0] > 00):
00: General purpose output push-pull
01: General purpose output Open-drain
10: Alternate function output Push-pull
11: Alternate function output Open-drain

Bits 29:28, 25:24, 21:20, 17:16, 13:12, 9:8, 5:4, 1:0 **MODEy[1:0]:** Port x mode bits (y= 8 .. 15)
These bits are written by software to configure the corresponding I/O port.
Refer to *Table 20: Port bit configuration table*.
00: Input mode (reset state)
01: Output mode, max speed 10 MHz.
10: Output mode, max speed 2 MHz.
11: Output mode, max speed 50 MHz.

9.2.3 Port input data register (GPIOx_IDR) (x=A..G)

Figure 9 - Configuration des broches PB8 à PB15. ST RM0008, p. 172.

9.2.4 Port output data register (GPIOx_ODR) (x=A..G)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy:** Port output data (y= 0 .. 15)
These bits can be read and written by software and can be accessed in Word mode only.
Note: For atomic bit set/reset, the ODR bits can be individually set and cleared by writing to the GPIOx_BSRR register (x = A .. G).

9.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy:** Port x Reset bit y (y= 0 .. 15)

Figure 10 - Registre de mise à 1 et de remise à 0. ST RM0008, p. 173.

La documentation du HD44780U montre le transfert en deux nibbles ainsi que les délais d'initialisation. Le schéma de séquence ci-dessous vient de la documentation du cours utilisée au Lab 1.

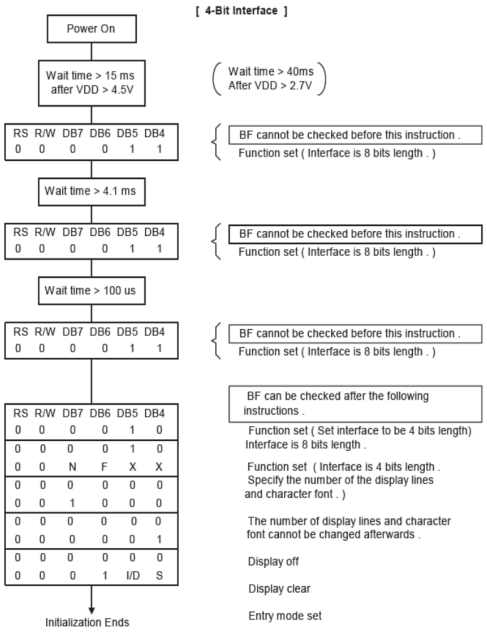


Figure 11 - Séquence de démarrage en mode 4 bits. Documentation du cours, Lab 1.

Interfacing to the MPU

The HD44780U can send data in either two 4-bit operations or one 8-bit operation, thus allowing interfacing with 4- or 8-bit MPUs.

- For 4-bit interface data, only four bus lines (DB4 to DB7) are used for transfer. Bus lines DB0 to DB3 are disabled. The data transfer between the HD44780U and the MPU is completed after the 4-bit data has been transferred twice. As for the order of data transfer, the four high order bits (for 8-bit operation, DB4 to DB7) are transferred before the four low order bits (for 8-bit operation, DB0 to DB3).

  The busy flag must be checked (one instruction) after the 4-bit data has been transferred twice. Two more 4-bit operations then transfer the busy flag and address counter data.
- For 8-bit interface data, all eight bus lines (DB0 to DB7) are used.

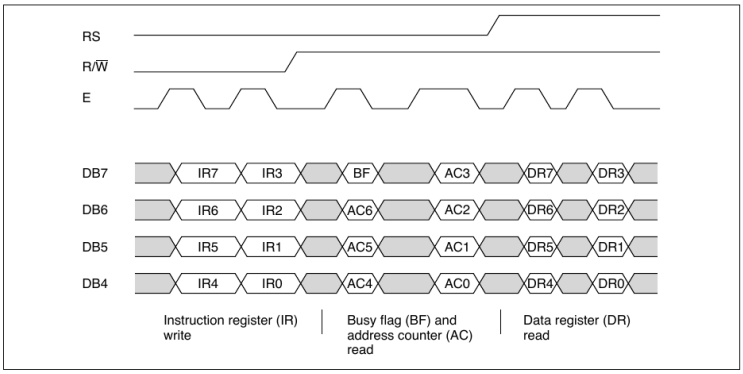


Figure 9 4-Bit Transfer Example

Figure 12 - Transfert et timing de l'interface 4 bits. Hitachi HD44780U, p. 22.

5.2 LCD_Commande()

Extrait du code source

```
void LCD_Commande(bool RW, bool RS, uint8_t Data)
{
    /* L'interface demandée envoie des octets; la lecture n'a pas de retour. */
    if (RW)
    {
        return;
    }

    LCD_SetPin(LCD_RS_PIN, RS);
    LCD_SetPin(LCD_RW_PIN, false); // mode écriture

    LCD_Envoyer_Nibble((uint8_t)(Data >> LCD_HIGH_NIBBLE_SHIFT)); // nibble haut en premier
    LCD_Envoyer_Nibble((uint8_t)(Data & LCD_DATA_MASK)); // puis nibble bas

    if ((!RS) && ((Data == LCD_CMD_CLEAR) || (Data == LCD_CMD_HOME)))
    {
        LCD_Delai_Ms(LCD_CLEAR_DELAY_MS); // ces deux commandes demandent plus de temps
    }
    else
    {
        LCD_Delai_Us(LCD_COMMAND_DELAY_US);
    }
}
```

Explication

`RS=0` sélectionne une commande ; `RS=1` sélectionne un caractère. Le pilote écrit toujours avec `R/W=0`. L'octet est séparé en deux groupes de quatre bits : le nibble de poids fort est transmis en premier, puis celui de poids faible. `LCD_Envoyer_Nibble()` place les quatre bits sur PB12 à PB15 et produit une impulsion sur PB11.

Le registre `GPIOB_BSRR` met chaque broche à 1 ou à 0. Pour éviter de toucher aux autres sorties, le code écrit les bits de mise à 1 ou de remise à 0 dans une seule écriture au registre. Les commandes `CLEAR` et `HOME` ont un délai de 2 ms ; les autres opérations attendent 50 µs.

Le tableau du hd44780 confirme l'ordre des nibbles dans une interface 4 bits. Pour `0x28`, le bus présente d'abord `0010`, puis `1000`.

Interfacing to the MPU

The HD44780U can send data in either two 4-bit operations or one 8-bit operation, thus allowing interfacing with 4- or 8-bit MPUs.

- For 4-bit interface data, only four bus lines (DB4 to DB7) are used for transfer. Bus lines DB0 to DB3 are disabled. The data transfer between the HD44780U and the MPU is completed after the 4-bit data has been transferred twice. As for the order of data transfer, the four high order bits (for 8-bit operation, DB4 to DB7) are transferred before the four low order bits (for 8-bit operation, DB0 to DB3).

  The busy flag must be checked (one instruction) after the 4-bit data has been transferred twice. Two more 4-bit operations then transfer the busy flag and address counter data.
- For 8-bit interface data, all eight bus lines (DB0 to DB7) are used.

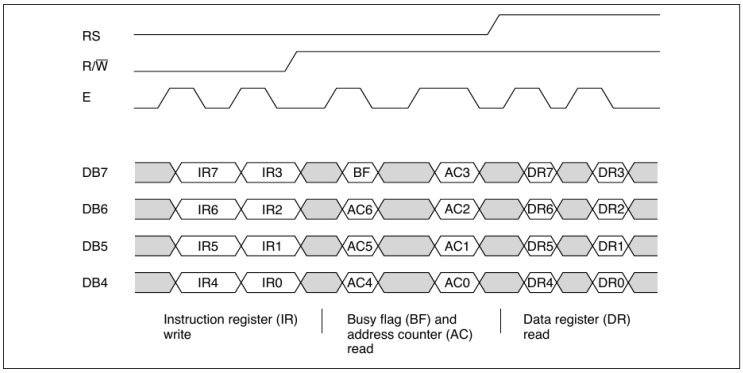


Figure 9 4-Bit Transfer Example

Figure 13 - Ordre des transferts en mode 4 bits. Hitachi HD44780U, p. 22.

5.3 LCD_Ecrire_Char()

Extrait du code source

```
void LCD_Ecrire_Char(unsigned char caractere, uint8_t ligne, uint8_t colonne)
{
    uint8_t adresse;

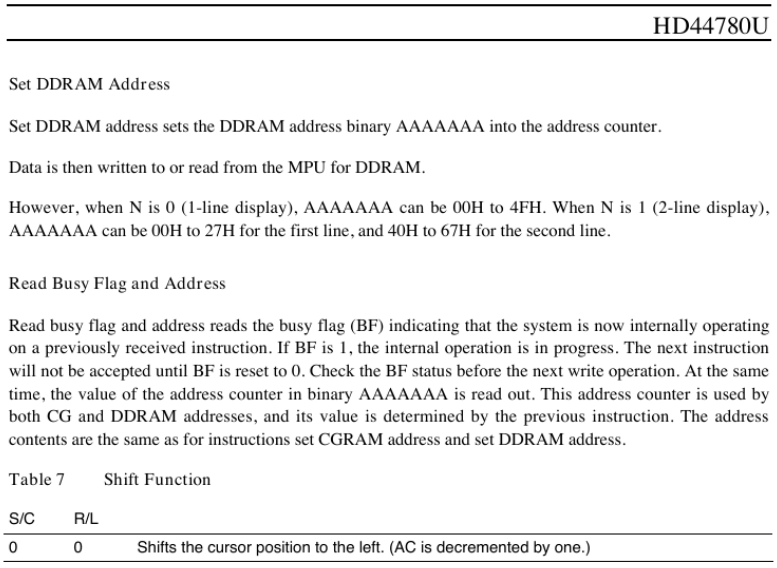
    /* Les lignes et colonnes sont numérotées à partir de 1. */
    if ((ligne < LCD_FIRST_POSITION) || (ligne > LCD_ROWS)
        || (colonne < LCD_FIRST_POSITION) || (colonne > LCD_COLUMNS))
    {
        return;
    }

    adresse = (uint8_t)(LCD_CMD_SET_DDRAM
                      | adresseLigne[ligne - LCD_FIRST_POSITION]
                      | (uint8_t)(colonne - LCD_FIRST_POSITION));
    LCD_Commande(false, false, adresse); // place le curseur
    LCD_Commande(false, true, (uint8_t)caractere); // envoie le caractère
}
```

Explication

Les numéros de ligne et de colonne commencent à 1. La fonction vérifie qu'ils sont dans les limites d'un écran 20 × 4. Elle ajoute le décalage de la colonne à l'adresse de base de la ligne. Les adresses utilisées sont `0x00`, `0x40`, `0x14` et `0x54`.

Le bit 7 de l'instruction est mis à 1 pour choisir une adresse DDRAM. La fonction envoie d'abord cette instruction avec `RS=0`, puis le caractère avec `RS=1`. Le tableau d'adresses correspond à la disposition mémoire du contrôleur pour un écran 20 × 4.



HD44780U

Set DDRAM Address

Set DDRAM address sets the DDRAM address binary AAAAAAA into the address counter.

Data is then written to or read from the MPU for DDRAM.

However, when N is 0 (1-line display), AAAAAAA can be 00H to 4FH. When N is 1 (2-line display), AAAAAAA can be 00H to 27H for the first line, and 40H to 67H for the second line.

Read Busy Flag and Address

Read busy flag and address reads the busy flag (BF) indicating that the system is now internally operating on a previously received instruction. If BF is 1, the internal operation is in progress. The next instruction will not be accepted until BF is reset to 0. Check the BF status before the next write operation. At the same time, the value of the address counter in binary AAAAAAA is read out. This address counter is used by both CG and DDRAM addresses, and its value is determined by the previous instruction. The address contents are the same as for instructions set CGRAM address and set DDRAM address.

Table 7 Shift Function

S/C	R/L	
0	0	Shifts the cursor position to the left. (AC is decremented by one.)

Figure 14 - Instruction et adresses DDRAM. Hitachi HD44780U, p. 29.

5.4 LCD_Affichage_Temps()

Extrait du code source

```
void LCD_Affichage_Temps(void)
{
    uint32_t temps;
    uint8_t heures;
    uint8_t minutes;
    uint8_t secondes;
    unsigned char caracteres[TIME_CHAR_COUNT];
    uint8_t colonne;

    temps = Timer2_GetSecondsSinceMidnight();
    heures = (uint8_t)(temps / SECONDS_PER_HOUR);
    minutes = (uint8_t)((temps / SECONDS_PER_MINUTE) % SECONDS_PER_MINUTE);
    secondes = (uint8_t)(temps % SECONDS_PER_MINUTE);

    caracteres[0] = (unsigned char)('0' + (heures / DECIMAL_BASE));
    caracteres[1] = (unsigned char)('0' + (heures % DECIMAL_BASE));
    caracteres[2] = ':';
    caracteres[3] = (unsigned char)('0' + (minutes / DECIMAL_BASE));
    caracteres[4] = (unsigned char)('0' + (minutes % DECIMAL_BASE));
    caracteres[5] = ':';
    caracteres[6] = (unsigned char)('0' + (secondes / DECIMAL_BASE));
    caracteres[7] = (unsigned char)('0' + (secondes % DECIMAL_BASE));

    for (colonne = 0U; colonne < TIME_CHAR_COUNT; colonne++)
    {
        LCD_Ecrire_Char(caracteres[colonne], LCD_FIRST_POSITION, (uint8_t)(colonne + LCD_FIRST_POSITION));
    }
}
```

Explication

La fonction lit le compteur de secondes, puis calcule les heures, les minutes et les secondes. Chaque chiffre est converti en caractère ASCII en ajoutant sa valeur à `'0'`. Elle envoie ensuite les huit caractères sur la première ligne, de la colonne 1 à la colonne 8.

Cette fonction ne configure pas de registre matériel elle-même. Les transferts se font par `LCD_Ecrire_Char()` et `LCD_Commande()`. La figure du fabricant ci-dessous montre comment les caractères passent au LCD par son interface 4 bits.

HD44780U

Interfacing to the MPU

The HD44780U can send data in either two 4-bit operations or one 8-bit operation, thus allowing interfacing with 4- or 8-bit MPUs.

- For 4-bit interface data, only four bus lines (DB4 to DB7) are used for transfer. Bus lines DB0 to DB3 are disabled. The data transfer between the HD44780U and the MPU is completed after the 4-bit data has been transferred twice. As for the order of data transfer, the four high order bits (for 8-bit operation, DB4 to DB7) are transferred before the four low order bits (for 8-bit operation, DB0 to DB3).
  The busy flag must be checked (one instruction) after the 4-bit data has been transferred twice. Two more 4-bit operations then transfer the busy flag and address counter data.
- For 8-bit interface data, all eight bus lines (DB0 to DB7) are used.

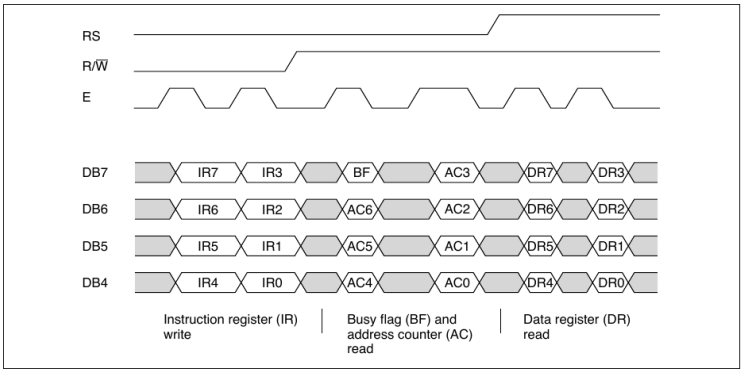


Figure 9 4-Bit Transfer Example

Figure 15 - Transfert d'un octet vers l'écran en mode 4 bits. Hitachi HD44780U, p. 22.

6. Réglage de TIM2

Le HSI est à 8 MHz. Comme le préscaler APB1 vaut 1, l'horloge de TIM2 est aussi à 8 MHz. Le code utilise `PSC=7999` et `ARR=999` :

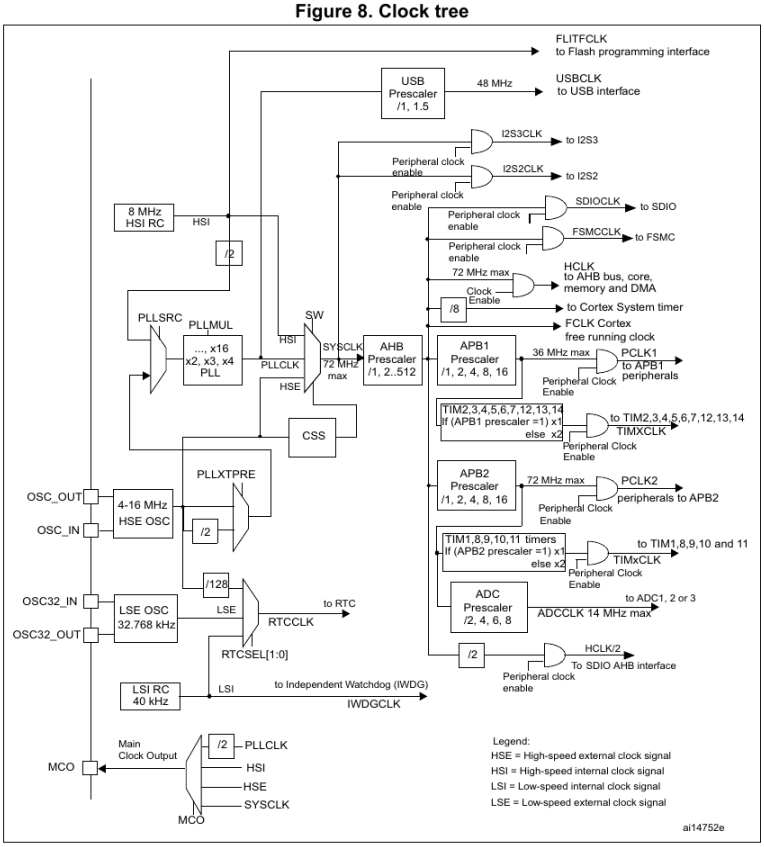
$$f_{\text{update}} = \frac{8\,000\,000}{(7999+1)(999+1)} = 1 \text{ Hz}$$

TIM2 produit donc une interruption chaque seconde. Le registre `TIM2_EGR` reçoit `UG` pour appliquer le préscaler. Le code efface ensuite les indicateurs en attente, active `UIE` dans `TIM2_DIER`, puis active l'IRQ 28 dans le NVIC.

Registre	Adresse	Valeur ou rôle
`RCC_APB1ENR`	`0x4002101C`	Bit 0 à 1 pour activer TIM2.
`TIM2_CR1`	`0x40000000`	`URS` choisit les demandes Update du compteur ; `CEN` démarre le timer.
`TIM2_DIER`	`0x4000000C`	`UIE` autorise l'interruption Update.
`TIM2_SR`	`0x40000010`	L'indicateur `UIF` signale une période terminée.
`TIM2_EGR`	`0x40000014`	`UG` applique immédiatement les valeurs de base du timer.
`TIM2_PSC`	`0x40000028`	Divise l'horloge par `PSC + 1 = 8000`.
`TIM2_ARR`	`0x4000002C`	Compte `ARR + 1 = 1000` impulsions avant Update.

Registre	Adresse	Valeur ou rôle
`NVIC_ISER0`	`0xE000E100`	Bit 28 à 1 pour activer l'IRQ TIM2.
`NVIC_ICER0`	`0xE000E180`	Bit 28 à 1 au début de l'initialisation pour désactiver temporairement l'IRQ TIM2.
`NVIC_ICPR0`	`0xE000E280`	Bit 28 à 1 pour effacer une ancienne IRQ TIM2 en attente.

Figure 8. Clock tree



1. When the HSI is used as a PLL clock input, the maximum system clock frequency that can be achieved is 64 MHz

Figure 16 - Arbre d'horloge STM32F103. ST RM0008, p. 93.

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **PSC[15:0]**: Prescaler value
The counter clock frequency CK_CNT is equal to f_{CK_PSC} / (PSC[15:0] + 1).
PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in “reset mode”).

15.4.12 TIMx auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0xFFFF

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **ARR[15:0]**: Prescaler value
ARR is the value to be loaded in the actual auto-reload register.
Refer to the *Section 15.3.1: Time-base unit* for more details about ARR update and behavior.
The counter is blocked while the auto-reload value is null.

15.4.13 TIMx capture/compare register 1 (TIMx_CCR1)

Address offset: 0x34

Reset value: 0x0000

Figure 17 - Définition du prescaler et du registre auto-reload de TIM2. ST RM0008, p. 419.

7. Conclusion

Dans ce laboratoire, j’ai programmé le STM32F103 pour afficher l’heure au format `HH:MM:SS` sur un écran LCD en mode 4 bits. TIM2 fait avancer l’heure chaque seconde.

Avec Saleae Logic 2 et le décodeur HD44780, j’ai vérifié que la commande `0x28` est envoyée, que les commandes de rafraîchissement reviennent environ chaque seconde et que les signaux montrent le passage de `23:59:59` à `00:00:00`.

Les trois cas de test sont réussis pour les signaux capturés sur le bus LCD. La période moyenne mesurée est de 998,802 ms, près de la seconde théorique. Ces essais ne confirment pas l’affichage visuel des caractères sur le module. Le branchement réel de la broche `E` et la compatibilité électrique entre les sorties 3,3 V du STM32 et l’entrée du LCD alimenté à 5 V restent à vérifier.

Références

— Consignes du laboratoire : `TEP_Prog_Lab3.pdf`, fourni avec le travail.
— Plan du laboratoire : `Laboratoires/Lab_3/plan.md`.
— Documentation du contrôleur : Hitachi, *HD44780U Dot Matrix Liquid Crystal Display Controller/Driver*, fichier `../../../Documentation/Datashit/HD44780.pdf`, pages 22 et 29.
— Documentation du module : Tinsharp, *TC2004A*, fichier `../../../Documentation/Datashit/TC2004A-01.pdf`, page 5.
— Documentation STM32 : STMicroelectronics, *RM0008 Reference Manual*, fichier RM0008 dans `../../../Documentation/Datashit/` pages 93, 113, 172-173 et 418-419.
— Rapport du Lab 1 : `../../../Documentation/RapportLab1.pdf` ; consulté pour repérer PB11 dans le code du Lab 1. Le câblage réel reste à confirmer.
— Rapport du Lab 2 : `Laboratoires/Lab_2/RapportLab2.md` ; utilisé comme référence pour la structure et la présentation du rapport.

Note sur l'utilisation de l'intelligence artificielle

Je tiens à préciser que les explications, les démarches, les résultats, les réflexions et les réponses aux questions de ce rapport viennent de moi. Je réalise également mon travail de programmation, même si j'utilise parfois l'IA pour obtenir des explications, des critiques ou des suggestions d'amélioration.

Je pense qu'il est important de rester prudent avant de conclure qu'un travail a été généré par l'IA simplement parce que son style « ressemble à de l'IA » ou parce qu'un détecteur l'indique. Ces outils ne sont pas infaillibles et peuvent produire des faux positifs. Un doute devrait donc servir à commencer une vérification, et non à constituer une preuve à lui seul.

Ce problème est d'ailleurs documenté. Une étude de Stanford a montré que certains détecteurs classaient à tort, en moyenne, **61,22 % des textes d'étudiants non anglophones** comme étant générés par l'IA. OpenAI avait également retiré son propre détecteur en juillet 2023 en raison de sa faible précision : il identifiait correctement seulement **26 % des textes générés par IA** lors de ses évaluations.

L'IAG évolue extrêmement rapidement et, même pour un utilisateur expérimenté ou un *power user*, il devient difficile de déterminer avec certitude l'origine d'un texte simplement en l'observant.

Transparence sur ma façon de travailler

— **Mise en page et Markdown :** c'est principalement pour cette partie que j'utilise l'IA. Je lui fournis moi-même mon code, mes explications, mes résultats, mes photos, mes captures d'écran et les autres textes. Elle s'occupe ensuite de les organiser, de les placer et d'écrire le Markdown nécessaire pour obtenir un rapport bien structuré et un PDF propre. Je connais également les bases du Markdown, mais cette méthode me fait gagner beaucoup de temps.
— **Mise à jour des rapports :** lorsque j'apporte des modifications finales à mon code, je peux demander à l'IA d'actualiser mon rapport. Je lui précise de **ne modifier aucunement mes explications, mes résultats ou mes autres textes personnels**, mais seulement de mettre à jour les éléments concernés.
— **Programmation et critiques :** j'utilise parfois l'IA pour comprendre certaines notions, repérer des erreurs ou obtenir des suggestions pour améliorer mon code, par exemple en remplaçant des *magic numbers* par des constantes plus explicites. J'écoute toujours ses critiques, mais je prends le temps de comprendre les modifications proposées avant de décider de les appliquer. Je sais également que l'IA peut se tromper.
— **Correction du français :** j'utilise **Antidote 12** pour corriger mes fautes et améliorer certaines formulations lorsque mes phrases sont difficiles à comprendre.

Je fais donc une distinction entre **utiliser l'IA comme outil d'apprentissage, de révision et de mise en forme, et lui demander de faire mon travail à ma place**.

Si quelque chose vous semble douteux, je préfère que vous veniez me voir avant de tirer une conclusion. Je suis disponible pour vous montrer mes fichiers de travail, mon historique de modifications ou vous expliquer en personne n'importe quelle partie de mon code ou de ma démarche.

Cette approche me semble plus juste et cohérente avec les principes de la PIEA, notamment l'importance d'un jugement d'évaluation fondé, documenté et interprété avec prudence.